

COMP 356
Programming Language Structures
Notes for Chapter 1 of *Concepts of Programming Languages*
Introduction

Why study programming languages?

- to make learning new languages easier
- to help in choosing an appropriate language for a particular application
- for reasons related to implementation issues:
 - learning techniques that are useful in other kinds of programs (compilers)
 - debugging tricky code (i.e. returning a pointer to a local variable)
 - writing efficient code (macros, inline functions)
- for reasons related to programming paradigms:
 - learning more ways of thinking about problem solving and programming
 - learning more languages (exemplifying paradigms)

1 Programming/Application Domains

- Scientific Computing
 - mathematical modeling and engineering design applications
 - the first application domain for computers and programming languages
 - characterized by:
 - * floating point computations
 - * array, matrix manipulations
 - * need for efficiency (number crunching)
 - languages:
 - * Fortran
 - * ALGOL 60
 - * parallel machine architectures and programming languages
- Business Applications
 - information processing and reporting applications
 - characterized by:
 - * report generation facilities
 - * decimal number manipulations (\$)
 - * database interoperability
 - * accessibility to nonprogrammers
 - languages:
 - * COBOL
 - * also affected design of Visual Basic and 4GLs (Delphi, PL/SQL)
- Artificial Intelligence
 - manipulating symbols instead of numbers

- characterized by:
 - * list (not array) processing
 - * dynamic typing
 - * self-modifying code
 - * flexibility
- example applications:
 - * natural language processing
 - * theorem proving
 - * automated integration & differentiation
- languages:
 - * LISP (and other functional languages)
 - * Prolog
- Systems Programming
 - developing operating systems, compilers, database systems, ...
 - characterized by:
 - * emphasis on efficiency
 - * ability to interface with hardware (low-level)
 - * few safety checks/restrictions
 - languages:
 - * C (for implementing Unix)
 - * other specialized languages
- Scripting Languages
 - controlling/adapting applications
 - characterized by:
 - * libraries or APIs for communication with external applications
 - * implemented using interpreters
 - * dynamic typing (for flexibility)
 - gaining rapidly in popularity
 - languages (with applications):
 - * JavaScript & VB Script (Web pages)
 - * Unix shell scripting languages (“assembling” Unix commands and applications into one program)
 - * TCL (assembly of reusable components) & Tk (GUIs)
 - * Perl (CGI scripting)
 - * awk (report generation)

2 Language Criteria/Design Principles

- simplicity
 - Affected by:
 - the number of language features (keywords, control structures, types)
 - the number of ways to do the same thing:
 - * pointers vs. references in C++

- * increments in C
- orthogonality
 - are language features independent, or do they interfere with each other? (no special cases)
 - examples:
 - * can data of all types be passed by value and by reference?
 - * can data of all types be passed to and returned from functions?
- reliability
 - does the language support writing reliable programs?
 - features affecting reliability:
 - * pointers/dynamic memory management
 - * aliasing
 - * syntax (= vs. ==)
 - * strength of type checking
 - * availability of exception handling
- efficiency
- portability
- support for (data) abstraction
- data types available
- control structures available

These “simple” criteria influence more general criteria such as:

- ease of learning the language
- ease of reading and writing programs (plus maintenance)
- cost of developing programs in the language

Many of the criteria are conflicting:

- adding data types often makes writing programs easier, but makes the language more complex and difficult to learn
- exception handling increases reliability, but adds complexity
- data abstraction can reduce efficiency

Language design (and selection) is always a balancing act.

3 Language Paradigms

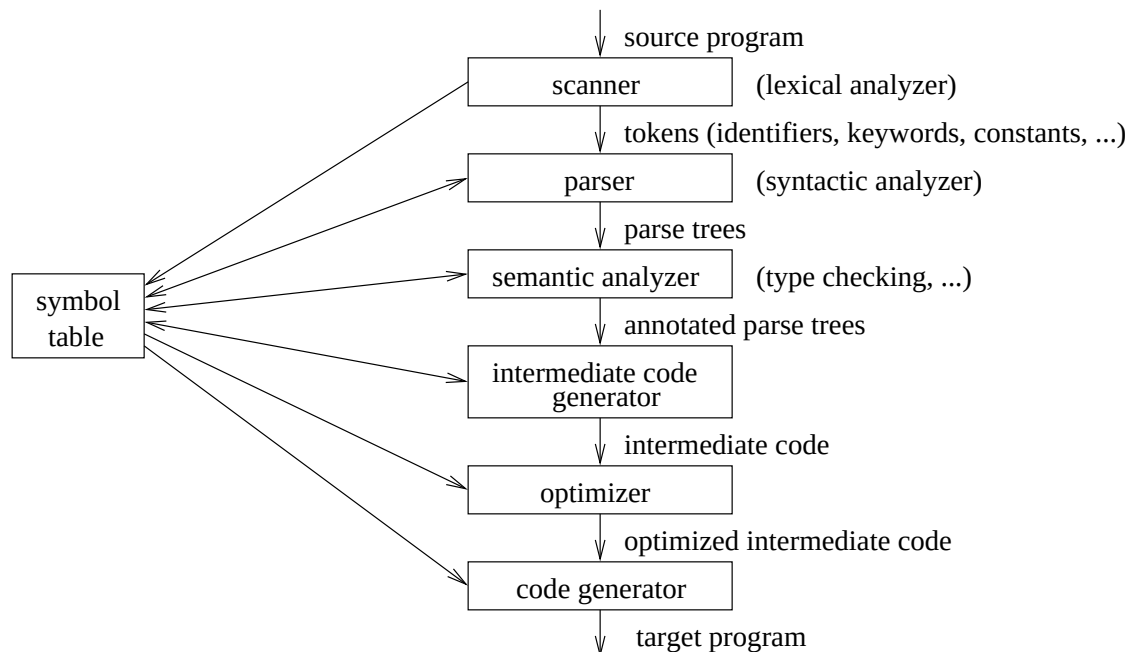
- imperative
 - based on assignment statements
 - example languages: Pascal, C, Basic
- object-oriented
 - developed from imperative languages

- based on program components (objects) that control their own state and operations
- typically include inheritance and polymorphism
- example languages: C++, Java, C#, Ada 95, Smalltalk
- functional
 - based on building and applying (calling) functions
 - no assignment statements or loops
 - example languages: LISP, Scheme, SML
- logic
 - based on logic and theorem proving
 - programmers specify properties of the result, rather than providing an algorithm to compute the result
 - example languages: Prolog

4 Implementation of Programming Languages

A **compiler** is a program that translates from one language (the **source** language) to another (the **target** language).

Typical stages (components) of a compiler:



If the target language is assembly (typical), it is then assembled and linked so that it is ready to be executed.

An **interpreter** is a program that directly executes (runs) a program written in the interpreted language. An interpreter is like a computer whose machine language is the interpreted language.

Some hybrid systems combine a compiler and an interpreter (older Java implementations, SML, some Smalltalks).

compiler	interpreter
reads source program once	reads source program each time it is run
detects “static” errors (syntax errors, type errors) at compile time	all error detection done at runtime
requires recompilation for each change to the source program	changed source is just interpreted again
allows optimization	optimization not worthwhile
leads to efficient execution	10 to 100 times slower

Table 1: Compilation vs. Interpretation